

Object Detection

1. Experiment Objective

Use the OpenCV DNN module to load the SSD MobileNet V2 pre-trained model and detect common objects in the camera image (COCO 90 classes).

2. Experiment Principle

1. Key Terms

Term	Description
Object Detection	Simultaneously locates and classifies multiple objects in an image, outputting bounding boxes + class labels + confidences
SSD	Single Shot MultiBox Detector; completes detection in a single forward inference pass without candidate region extraction
MobileNet V2	A lightweight convolutional neural network designed for mobile and embedded devices, balancing accuracy and speed
COCO Dataset	A large-scale object detection dataset containing 90 classes of common objects (people, cars, animals, furniture, etc.)
Confidence	The model's degree of certainty for a detection result, ranging from 0 to 1; only results > 0.4 are kept to filter out false detections
Blob	The standard input format for neural networks; an image preprocessed by mean subtraction (104, 117, 123) and resizing (300×300)

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start object detection
```

Data Flow: ESP32 camera → /camera/image_raw → image conversion → `b1obFromImage` converts to a 300×300 blob → SSD MobileNet V2 forward inference → filter by confidence > 0.4 → draw bounding boxes + labels → `cv2.imshow` real-time display.

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	☒ Supported
No-PTZ Version	☒ Supported
Required Peripherals	ESP32 camera

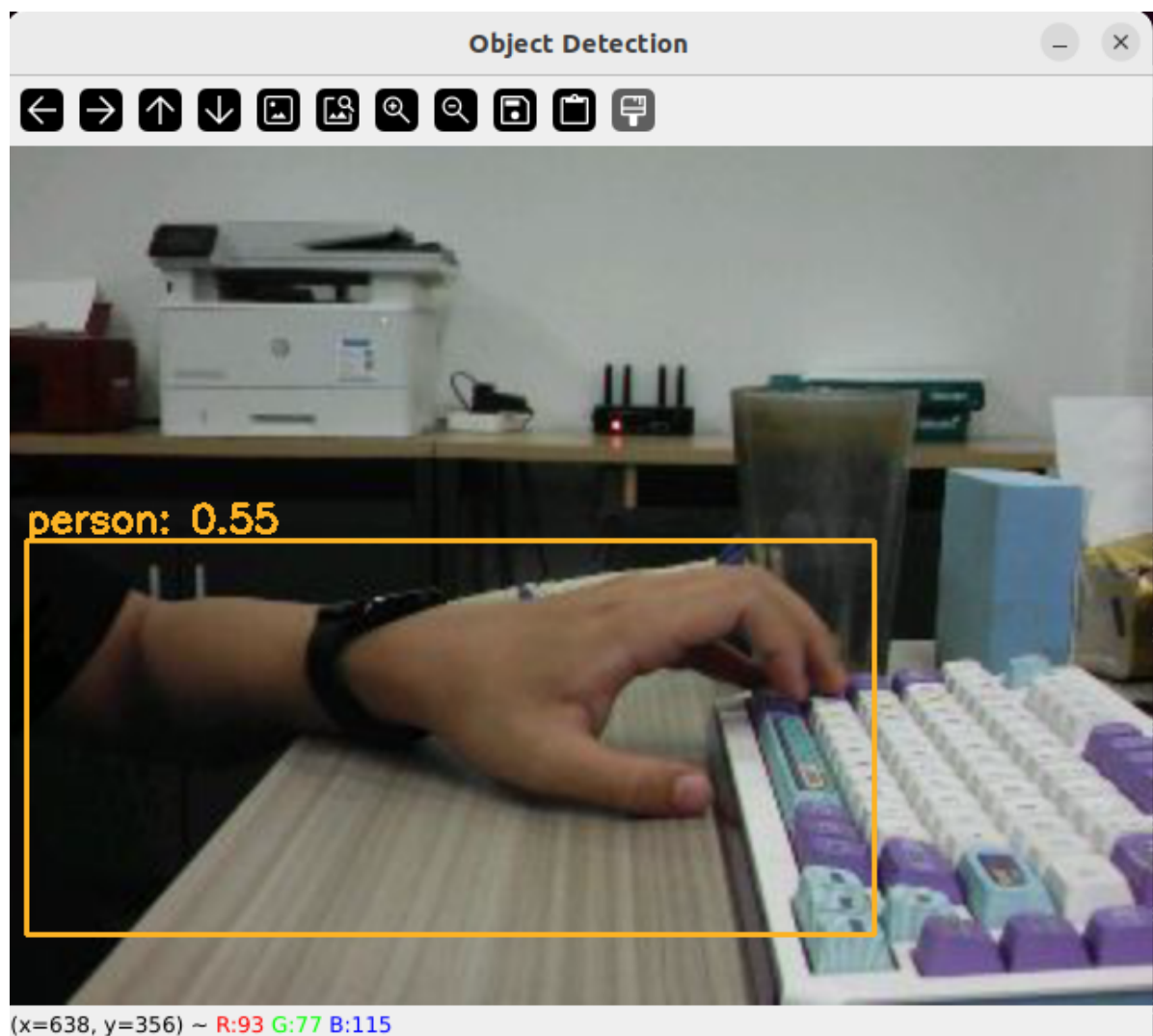
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboongyahboon:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboon/.ros/log/2026-07-28-10-47-55-372666-yahboon-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_descriptio
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_Link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Object Detection (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg detect_object.launch.py
```



4. Operation Instructions

Workflow: Subscribe to `/camera/image_raw` → image conversion → `blobFromImage` (300×300, mean subtraction) → SSD inference → filter by confidence > 0.4 → draw a bounding box (`cv2.rectangle`) + `class name: confidence` label (`cv2.putText`) for each detection → display with `cv2.imshow`.

Item	Details
Model architecture	SSD + MobileNet V2 (TensorFlow)
Input size	300×300
Training dataset	COCO
Number of classes	90 classes
Confidence threshold	0.4

Interaction: Press `q` to exit.

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **Normal detection:** Objects are enclosed by colored rectangles, with `class name: confidence` displayed above the box (e.g. `person: 0.95`)
- **Multiple objects:** Each detected object is drawn independently; objects of the same class share a uniform color
- **No-object scene:** All detection box confidences are ≤ 0.4 , showing the raw image without annotation



5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_object.launch`

h.py

- Node:

```
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_
pkg/detect_object.py
```

1. Core Node Analysis

`ObjectDetectionNode` loads the SSD MobileNet V2 model, performs object detection on every frame, and visualizes the results.

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/de
tect_object.py
class ObjectDetectionNode(Node):
    def __init__(self):
        super().__init__('object_detection_node')
        self.bridge = CvBridge()
        topic_name = '/camera/image_raw'

        self.subscription = self.create_subscription(
            Image, topic_name, self.image_callback,
            QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=1,
                        reliability=QoSReliabilityPolicy.BEST_EFFORT,
                        durability=QSDurabilityPolicy.VOLATILE))

        # Load the COCO class list file
        config_path = ('../microros_v2_opencv_visual_pkg/config')
        with open(f'{config_path}/object_detection_coco.txt', 'r') as f:
            self.class_names = f.read().split('\n')

        self.COLORS = np.random.uniform(0, 255, size=(len(self.class_names), 3))

        # Load the SSD MobileNet V2 model
        self.model = cv.dnn.readNet(
            model=f'{config_path}/frozen_inference_graph.pb',
            config=f'{config_path}/ssd_mobilenet_v2_coco.txt',
            framework='TensorFlow')

    def target_detection(self, image):
        image_height, image_width, _ = image.shape
        blob = cv.dnn.blobFromImage(
            image=image, size=(300, 300), mean=(104, 117, 123), swapRB=True)
        self.model.setInput(blob)
        output = self.model.forward()

        for detection in output[0, 0, :, :]:
            confidence = detection[2]
            if confidence > 0.4:
                class_id = int(detection[1])
                # Bounding box coordinate denormalization
                box_x = int(detection[3] * image_width)
                box_y = int(detection[4] * image_height)
                box_w = int(detection[5] * image_width)
                box_h = int(detection[6] * image_height)
                # Draw bounding box + label
                cv.rectangle(image, (box_x, box_y), (box_x+box_w, box_y+box_h),
```

```

                self.COLORS[class_id], 2)
        label = f'{self.class_names[class_id-1]}: {confidence:.2f}'
        cv.putText(image, label, (box_x, box_y-5),
                    cv.FONT_HERSHEY_SIMPLEX, 0.7, self.COLORS[class_id],
2)

        return image

```

Key logic notes:

- `blobFromImage` with `swapRB=True`: OpenCV uses BGR by default while the model expects RGB; this parameter performs the conversion automatically
- Mean subtraction `(104, 117, 123)`: subtracts the ImageNet training set mean from each channel to normalize the input distribution
- `output[0, 0, :, :]`: the SSD output tensor has the shape `[1, 1, N, 7]`, each row being `[_ , class_id, confidence, x1, y1, x2, y2]`
- Coordinate denormalization: the model outputs relative coordinates in the 0~1 range; multiplying by the actual image width and height yields pixel coordinates

2. Key Parameters

Parameter definition files:

- Launch:


```
~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_object.launch
h.py
```
- Node:


```
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_
pkg/detect_object.py
```

Parameter	Type	Default Value	Description
<code>topic_name</code>	string	<code>/camera/image_raw</code>	Input image topic
<code>confidence_threshold</code>	float	<code>0.4</code>	Detection confidence threshold
<code>blob_size</code>	tuple	<code>(300, 300)</code>	Model input blob size
<code>blob_mean</code>	tuple	<code>(104, 117, 123)</code>	Blob mean subtraction parameters (B, G, R)
<code>num_classes</code>	int	<code>90</code>	Total number of COCO dataset classes

3. Node Description

Node	Function
<code>object_detection_node</code>	Subscribes to raw images, runs SSD MobileNet V2 inference for COCO 90-class objects, and visualizes with OpenCV

6. Frequently Asked Questions

Problem	Cause	Solution
Slow detection	The model is large	Use a lightweight model (e.g. MobileNet-SSD)
Many false detections	Confidence threshold too low	Increase the <code>confidence_threshold</code> parameter